

Tổng Hợp Kiến Thức

1. Khái niệm cốt lõi

- **Authentication (AuthN):** Xác định "Bạn là ai?". Verify danh tính user.
- **Authorization (AuthZ):** Xác định "Bạn được làm gì?". Kiểm tra quyền truy cập sau khi AuthN. **Luôn chạy sau AuthN.**
- **bcrypt** : Thư viện hash password an toàn (tự động salt).
 - `bcrypt.hash(plainPass, saltRounds)` : Tạo hash để lưu vào DB.
 - `bcrypt.compare(plainPass, hashedPassFromDB)` : So sánh khi login.
- **JWT (JSON Web Token):** Chuẩn token stateless để duy trì phiên đăng nhập. Gồm 3 phần: Header, Payload, Signature.
- **Payload Strategy (Bài này):** Chỉ chứa `userId` . Thông tin khác (role, username) được **query từ DB** trong middleware xác thực.
 - **Ưu điểm:** Dữ liệu user (đặc biệt là role) luôn mới nhất.
 - **Nhược điểm:** Tăng tải DB trên mỗi request cần xác thực.
- **RBAC (Role-Based Access Control):** Phân quyền dựa trên vai trò (vd: 'user', 'admin').
- **Middleware:** Các hàm trung gian xử lý request/response (AuthN, AuthZ, Validation, Logging...). **Thứ tự rất quan trọng!**

2. Workflow & Code Snippets

2.1. Đăng ký (Registration)

- **Route:** `POST /api/auth/register`
- **Controller (`authController.js`):**

```
const hashedPassword = await bcrypt.hash(password, 10); // Hash pass
const newUser = await User.create({ username, email, password: hashedPassword }); // Role 'user' tự gán
const userResponse = newUser.toJSON(); // Loại bỏ pass nhờ defaultScope
res.status(201).json({ message: '...', user: userResponse });
```

- **Validation (`routes/authRoutes.js`):** Dùng `express-validator` (`body('username')...` , `body('email')...` , `body('password')...`)

2.2. Đăng nhập (Login)

- **Route:** `POST /api/auth/login`
- **Controller (`authController.js`):**

```
// Tìm user (kèm pass) bằng email hoặc username
const user = await User.scope('withPassword').findOne({ where: { [Op.or]: [{ email: emailOrUsername }, { username: emailOrUsername }] } });
if (!user) { /* 401 */ }
const isMatch = await bcrypt.compare(password, user.password); // So sánh pass
if (!isMatch) { /* 401 */ }

// Tạo JWT chỉ với userId
const payload = { userId: user.id };
const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRES_IN });

const userResponse = user.toJSON(); delete userResponse.password; // Lấy info trả về client
res.json({ message: '...', token: token, user: userResponse });
```

- **Validation (`routes/authRoutes.js`):** Dùng `express-validator` (`body('emailOrUsername')...` , `body('password')...`)

2.3. Xác thực Token (Authentication Middleware)

- **Middleware** (`middlewares/authenticateToken.js`):

```
const jwt = require('jsonwebtoken');
const { User } = require('../models');

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Token required.' });

  jwt.verify(token, process.env.JWT_SECRET, async (err, decodedPayload) => {
    if (err) { // Handle JWT errors (expired, invalid signature)
      return res.status(err instanceof jwt.TokenExpiredError ? 401 : 403).json({ message: err.message });
    }
    if (!decodedPayload || !decodedPayload.userId) {
      return res.status(403).json({ message: 'Invalid token payload.' });
    }

    try {
      const user = await User.findByPk(decodedPayload.userId); // Query DB
      if (!user) return res.status(401).json({ message: 'User not found.' });

      req.user = user; // Attach DB user object to request
      next();
    } catch (dbError) { next(dbError); } // Handle DB errors
  });
};
module.exports = authenticateToken;
```

- **Áp dụng:** Đặt trước các route/controller cần đăng nhập.

```
// routes/someRoute.js
const authenticateToken = require('../middlewares/authenticateToken');
router.post('/protected', authenticateToken, someController.handleProtected);
router.get('/me', authenticateToken, authController.getMe);
```

2.4. Phân quyền (Authorization Middleware - RBAC)

- **Middleware** (`middlewares/authorizeRole.js`):

```
const authorizeRole = (allowedRoles) => {
  const roles = Array.isArray(allowedRoles) ? allowedRoles : [allowedRoles];
  return (req, res, next) => {
    // !! Quan trọng: authenticateToken phải chạy trước !!
    if (!req.user || !req.user.role) {
      return res.status(403).json({ message: 'Cannot determine user role.' });
    }
    if (!roles.includes(req.user.role)) { // Kiểm tra role từ req.user (đã lấy từ DB)
      return res.status(403).json({ message: 'Access forbidden: Insufficient permissions.' });
    }
    next(); // Role hợp lệ
  };
};
module.exports = authorizeRole;
```

- **Áp dụng:** Đặt **SAU** `authenticateToken` và **TRƯỚC** controller.

```
// routes/adminRoute.js
const authenticateToken = require('../middlewares/authenticateToken');
const authorizeRole = require('../middlewares/authorizeRole');
router.delete('/resource/:id',
  authenticateToken, // 1. Verify token & Get user from DB
  authorizeRole('admin'), // 2. Check if user.role is 'admin'
  adminController.deleteResource // 3. Handle request
);
router.put('/resource/:id',
  authenticateToken,
  authorizeRole(['admin', 'editor']), // Cho phép nhiều role
  someController.updateResource
);
```

3. Thứ tự Middleware (Quan trọng!)

Luồng xử lý cho route cần cả AuthN và AuthZ:

1. `authenticateToken` (Verify JWT, Query DB, Gắn `req.user`)
2. `authorizeRole` (Kiểm tra `req.user.role`)
3. (Tuỳ chọn) `validationMiddleware` (Kiểm tra `req.body` , `req.params` , etc.)
4. `controllerFunction` (Xử lý logic chính)